

MADRS

Security Platform

نظام كشف الأنشطة الخبيثة والاستجابة لها

Malicious Activity Detection & Response System

الوثيقة الفنية الشاملة — شرح كامل لجميع مكونات النظام وواجهاته

الإصدار 3.0 | 2026 | وثيقة فنية

الفهرس العام

الجزء الأول — دليل النظام والواجهات

مقدمة عن النظام · المعمارية العامة · دورة الحياة

قاعدة البيانات · RBAC · Firewall

Backend · ML Service

واجهة (index, login, dashboard, alerts, scan, monitoring, incidents, firewall)

شرح 21

training, analytics, MITRE, reports, response, classification, users, audit, webhooks,

(notifications, settings, contact

واجهات API	الرئيسية · سيناريو Demo	· التشغيل والإعداد
------------	-------------------------	--------------------

الجزء الثاني — التوثيق التقني العميق

Pipeline	الكامل · التقاط الترافيك (NFStream)
----------	-------------------------------------

· مرشحات الدقة	طبقات الكشف الثلاث: Safe-flow · XGBoost
----------------	---

ICMP Heuristics · Behavioral Baseline · MITRE Mapping

نقل الكشف Correlation (Incidents) · ML→Backend

بالفصيل)	· الحظر (nftables)	Persistence + Audit
----------	--------------------	---------------------

الحل الجماعي · الأرشفة · جدول كل الثوابت

1. مقدمة عن النظام

MADRS هو منصة متكاملة للأمن السيبراني تجمع بين كشف التهديدات باستخدام الذكاء الاصطناعي، والاستجابة التلقائية، وإدارة التنبيهات والحوادث، والتوثيق وإصدار التقارير.

يعمل النظام على تحليل حركة الشبكة (سواء حياً عبر بطاقة الشبكة أو من ملفات **PCAP** مسجلة)، ويستخدم نموذج تعلم آلي مُدرَّب لتصنيف كل تدفق شبكي إلى **سليم (Benign)** أو **هجوم (Attack)** مع تحديد نوع الهجوم ودرجة الخطورة ونسبة الثقة.

أبرز قدرات النظام:

- **كشف ذكي:** تصنيف التدفقات باستخدام نموذج **XGBoost** مع كشف الشذوذ السلوكي.

- **استجابة فعلية:** حظر عناوين IP على مستوى نواة النظام عبر **nftables** فيوقف الترافيك حقيقةً.

- **إدارة دورة حياة التنبيه:** جديد ← قيد التحقيق ← محلول / إيجابية خاطئة.

- **التعلم المستمر:** أرشفة التنبيهات المحلولة في بيانات تدريب لإعادة تدريب النموذج.

- **تعدد المنظّمات (Multi-tenant):** عزل بيانات كل منظمة عن الأخرى.

- **التوثيق والتقارير:** تقارير PDF و CSV احترافية.

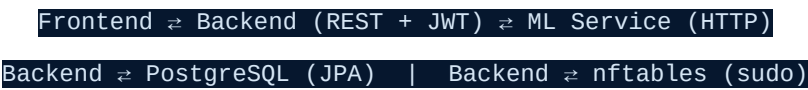
- **التكامل الخارجي:** Webhooks و Threat Intelligence (AbuseIPDB).

2. المعمارية العامة

يتكوّن النظام من أربعة مكوّنات رئيسية تعمل معاً:

المكوّن	التقنية	الوظيفة	المنفذ
Frontend	HTML / CSS / JavaScript	واجهات المستخدم والتفاعل	5500
Backend	Java / Spring Boot	منطق العمل، الـ API، الأمان، قاعدة البيانات	8080
ML Service	Python / FastAPI	التصنيف، المراقبة الحية، تحليل PCAP	8001
Database	PostgreSQL	تخزين كل البيانات	5432

كيف تتكامل المكوّنات؟



• تتواصل الواجهات مع الـ Backend عبر REST API مع توثيق JWT.

• يتطلب الـ Backend التصنيفات من الـ ML Service، ويخزّن النتائج في قاعدة البيانات.

• عند اتخاذ قرار الحظر، يطبّق الـ Backend قاعدة على nftables في نواة النظام.

3. دورة حياة الكشف والاستجابة

هذه الرحلة الكاملة لأي هجمة داخل النظام:

إنشاء تنبيه → تحقيق المحلل → قرار (حظر/حل) → أرشفة في التدريب → تقرير → ML كشف الترافيك → تصنيف

1.الكشف: تلتقط التدفقات حياً أو من ملف PCAP.

2.التصنيف: يحدّد النموذج النوع والخطورة ونسبة الثقة.

3.إنشاء التنبيه: يُحفظ تنبيه جديد إن تجاوز عتبة الثقة، ولم يكن مصدره محظوراً مسبقاً.

4.التحقيق: يفحص المحلل التفاصيل ويضيف ملاحظات وتغذية راجعة.

5.الاستجابة: حظر ال IP، أو حل التنبيه، أو وسمه كإيجابية خاطئة.

6.الحظر الجماعي: عند حظر IP، تُحلّ تلقائياً كل تنبيهاته المفتوحة وتُؤرشف.

7.الأرشفة: تُخزّن التنبيهات المحلولة في ml_training_data مع القرار والاستجابة.

8.التقرير: يُصدّر تقرير PDF/CSV للتوثيق.

ميزة مهمة: حل تنبيه واحد من 100 هجمة متكررة من نفس المصدر والنوع يحلّها جميعاً ويؤرشفها دفعة واحدة، بدل تكرار نفس العمل 100

مرة.

ميزة مهمة: عنوان IP المحظور لا يظهر مرة أخرى كتنبيه جديد، لأن النظام يتجاهل ترافيكه قبل إنشاء التنبيه.

4. المكوّن الخلفي (Backend / Spring Boot)

ال Backend هو قلب النظام، ومسؤول عن منطق العمل، الأمان، والتواصل مع قاعدة البيانات و ML Service. وهو مبني بلغة **Java** باستخدام إطار **Spring Boot**.

أهم المتحكّمات (Controllers):

الوظيفة	المتحكم
تسجيل الدخول والتسجيل وإصدار توكن JWT	AuthController
إدارة التنبيهات: قائمة، تفاصيل، تحديث حالة، تعليقات، تصدير	AlertController
رفع وتحليل ملفات PCAP	ScanController
التحكم في المراقبة الحية (تشغيل/إيقاف/حالة)	MonitorController
إدارة الحوادث (تجميعات التنبيهات)	IncidentController
حظر/فك حظر عناوين IP	FirewallController
بيانات تدريب النموذج وتصديرها CSV	MLTrainingController
مؤشرات الأداء والاتجاهات	AnalyticsController
إصدار تقارير PDF و CSV	ReportController
إدارة المستخدمين والمنظمات	UserController / OrganizationController

سجل التدقيق للإجراءات الحساسة

AuditLogController

إدارة تكاملات الإشعارات الخارجية

WebhookController

سمعة عناوين IP (AbuseIPDB)

ThreatIntelController

إعدادات المنصة

SettingsController

طلبات إنشاء حسابات المؤسسات وإرسال بريد الترحيب

CompanyRequestController

صحة النظام ومقاييسه

HealthController / MetricsController

أهم الخدمات (Services):

• **AlertService:** إنشاء/تحديث التنبيهات، تجاهل المحظورين، والحل الجماعي للتنبيهات المتشابهة.

• **BlockedIpService:** الحظر/فك الحظر، تطبيق قواعد nftables، وحل التنبيهات وأرشفتها عند الحظر.

• **MLTrainingService:** أرشفة التنبيهات المحلولة مع كامل السياق والقرار.

• **ScanService:** معالجة ملفات PCAP وتحويل التدفقات إلى تنبيهات.

• **LiveMonitorService:** جلب الكشوفات الحية من ML Service وتحويلها لتنبيهات.

• **ReportExportService:** توليد تقارير PDF.

• **ThreatIntelService:** الاستعلام عن سمعة عناوين IP.

• **CompanyRequestService:** إنشاء المنظمة والمستخدم المدير وإرسال بريد الترحيب.

5. خدمة الذكاء الاصطناعي (ML Service)

خدمة مستقلة مبنية بـ **Python / FastAPI**، مسؤولة عن كل ما يتعلق بالتعلم الآلي والتقاط الشبكة.

وظائفها:

- **تحميل النموذج:** نموذج **XGBoost** مُدرَّب مع قائمة الميزات (features).
- **التصنيف:** استقبال ميزات التدفق وإرجاع التصنيف ونسبة الثقة.
- **المراقبة الحية:** التقاط الحزم من بطاقة الشبكة، استخراج التدفقات، وتصنيفها على دفعات.
- **الكشف السلوكي:** قياس خط أساس للسلوك واكتشاف الشذوذ (مثل ICMP flood أو port scan).
- **تحليل PCAP:** قراءة الملف، استخراج التدفقات، والتصنيف.

كيف يعمل التقاط الشبكة؟

تحتاج الخدمة صلاحية التقاط الحزم الخام. يُمنح مفسر Python صلاحية **CAP_NET_RAW** عبر سكربت `scripts/grant-capture-perms.sh` ليُعمل الالتقاط دون تشغيل كامل بصلاحية الجذر.

ملاحظة أداء: لتسريع تحليل ملفات PCAP الكبيرة، يكون **SHAP** وعيّنات الحمولة معطّلة افتراضياً، مع الحفاظ على الإحصاءات والتفسير

المبسّط للتنبيهات.

6. قاعدة البيانات (PostgreSQL)

قاعدة البيانات هي `ai_ids`، والمستخدم `aisec`. يُهيأ المخطط عبر ملف `db/init.sql`.

أهم الجداول:

المحتوى	الجدول
التنبيهات: النوع، المصدر، الخطورة، الثقة، الحالة، الوصف	<code>alerts</code>
تعليقات وملاحظات التحقيق على التنبيهات	<code>alert_comments</code>
عناوين IP المحظورة وسبب ومصدر الحظر	<code>blocked_ips</code>
التنبيهات المؤرشفة لإعادة تدريب النموذج مع القرار والاستجابة	<code>ml_training_data</code>
الحوادث (تجميع تنبيهات مترابطة)	<code>incidents</code>
المستخدمون والمنظمات	<code>users / organizations</code>
سجل الإجراءات الحساسة	<code>audit_log</code>
سمعة عناوين IP المخزنة مؤقتاً	<code>ip_reputation</code>

7. الجدار الناري (Firewall / nftables)

عند حظر عنوان IP، لا يكتفي النظام بتسجيله في قاعدة البيانات، بل يطبق قاعدة فعلية على نواة لينكس عبر `nftables` لإسقاط ترافيك ذلك العنوان قبل وصوله لأي خدمة.

كيف يعمل؟

- يُنشأ جدول `inet aisec` يحتوي مجموعتين: `aisec_block_v4` و `aisec_block_v6`.

- تُطبّق قواعد `drop` على ثلاث سلاسل: `input` (الوارد)، `output` (الصادر)، `forward` (المُمرّر).

- عند الحظر، يُضاف العنوان للمجموعة المناسبة فيُسقط ترافيكه فوراً.

متطلبات التفعيل:

```
sudo tee /etc/sudoers.d/aisec-nft <<'EOF'
mohamed ALL=(root) NOPASSWD: /usr/sbin/nft
EOF
sudo chmod 0440 /etc/sudoers.d/aisec-nft
```

هام: بدون هذه الصلاحية يعمل النظام بوضع تسجيل في قاعدة البيانات فقط (NoOp) دون إيقاف فعلي للترافيك.

8. الأمان والصلاحيات (Security & RBAC)

•المصادقة: عبر JWT يُصدر عند تسجيل الدخول ويُرفق مع كل طلب.

•تشفير كلمات المرور: باستخدام خوارزمية تجزئة آمنة قبل التخزين.

•عزل المنظمات: كل مستخدم يرى بيانات منظمته فقط.

•سجل التدقيق: تسجيل كل عملية حساسة (دخول، حظر، تعديل تنبيه).

الدور	يرى	يعدل	صفحات خاصة
ADMIN	كل شيء	كل شيء	Settings, Audit, Webhooks
ORG_ADMIN	منظمته	مستخدمي	Audit, Webhooks
N	فقط	منظمته	
ANALYST	منظمته	التنبيهات	—
	فقط	والحوادث	
VIEWER	منظمته	قراءة فقط	—
	فقط		

9. شرح تفصيلي لكل الواجهات

فيما يلي شرح مفصل لكل صفحة في النظام: الغرض منها، وأهم عناصرها، والإجراءات المتاحة فيها.

9.1 الصفحة التعريفية (index.html)

الغرض: صفحة الهبوط التعريفية بالمنصة.

• **Hero:** عنوان رئيسي يعرّف بـ MADRS وقدراته.

• **الإحصاءات:** أرقام تعريفية عن النظام.

• **الأقسام:** الميزات، كيف يعمل، الدعوة للإجراء.

• **Login:** ينقل لصفحة تسجيل الدخول.

9.2 تسجيل الدخول (login.html)

الغرض: الدخول للمنصة عبر بيانات الاعتماد.

• **Username / Password:** اسم المستخدم وكلمة المرور (مع إظهار/إخفاء).

• **Remember me:** حفظ الجلسة في المتصفح.

• **Forgot Password:** الانتقال لاستعادة كلمة المرور.

• **Sign In:** إرسال البيانات للتحقق وإصدار JWT.

• **Request enterprise access:** نموذج طلب حساب مؤسسي.

9.3 استعادة كلمة المرور (forgot-password / reset-password)

الغرض: استعادة الوصول عند نسيان كلمة المرور.

• **forgot-password**: إدخال البريد لإرسال رابط الاستعادة.

• **reset-password**: تعيين كلمة مرور جديدة عبر الرابط.

9.4 لوحة التحكم (dashboard.html)

الغرض: نظرة عامة فورية على حالة الأمان.

• **KPIs**: التنبيهات النشطة، التهديدات المحظورة، صحة النظام، متوسط وقت الاستجابة.

• **Threat Activity**: رسم بياني للهجمات والحجب (24 ساعة / 7 / 30 يوم).

• **Attack Types**: رسم دائري لتوزيع أنواع الهجمات.

• **Recent Alerts / Activity Log**: أحدث التنبيهات وسجل النشاط.

• **Notifications**: الإشعارات مع تعيين الكل كمقروء.

9.5 التنبيهات (alerts.html)

الغرض: إدارة والتحقق في كل تنبيهات الأمان.

• **Severity Pills**: عدّاد لكل مستوى: Critical / High / Medium / Low.

• **أدوات**: بحث، تصفية حسب الخطورة، ترتيب، تصدير CSV، إظهار/إخفاء أعمدة.

• **إجراءات جماعية**: Investigate, Resolve, False Positive, Clear.

• **الجدول**: المعرف، النوع، الخطورة، المصدر، الحالة، المسؤول، الوقت، الإجراءات.

• **نافذة التفاصيل**: تعيين لمحلل، تغذية راجعة لـ ML، ملاحظات التحقيق، وأزرار تغيير الحالة.

عند حل تنبيه أو وسمه إيجابية خاطئة، يُؤرشف تلقائياً في بيانات التدريب، وتُحلّ معه التنبيهات المتشابهة من نفس المصدر والنوع.

9.6 مسح PCAP (scan.html)

الغرض: رفع ملفات الشبكة المسجلة لتحليلها بالذكاء الاصطناعي.

• **منطقة الرفع:** سحب وإفلات أو تصفّح. الصيغ: .cap / .pcapng / .pcap.

• **Min Confidence:** أقل نسبة ثقة تُقبل لتحويل النتيجة إلى تنبيه.

• **Limit per run:** حدّ عدد التدفقات المعالجة.

• **التقدم:** رفع ← تحليل AI ← حفظ التنبيهات.

• **النتائج:** إجمالي التدفقات، السليمة، الهجمات، متوسط الثقة، وجدول الهجمات المكتشفة.

9.7 المراقبة الحية (monitoring.html)

الغرض: مراقبة الشبكة وصحة النظام لحظياً.

• **اختيار البطاقة:** Network Interface وأزرار Start / Stop.

• **المؤشرات الحية:** إجمالي التدفقات، السليمة، الهجمات، تدفقات/ثانية، هجمات/ثانية، عمق الطابور.

• **Sparkline:** ترافيك آخر 60 ثانية.

• **Recent Detections:** أحدث الكشفوفات.

• **حالة الخدمات:** CPU، الذاكرة، الشبكة، القرص، وحالة كل خدمة.

9.8 الحوادث (incidents.html / incident.html)

الغرض: عرض الحوادث كتجميعات لتنبيهات مترابطة.

• **incidents:** قائمة الحوادث مع البحث والتصفية حسب الحالة.

• **بطاقة الحادث:** العنوان، IP المصدر، عائلة الهجوم، عدد التنبيهات، الحالة، الوقت.

• **incident:** صفحة تفاصيل حادث واحد مع تنبيهاته وخط زمنه.

9.9 الجدار الناري (firewall.html)

الغرض: إدارة عناوين IP المحظورة.

• **قائمة المحظورين:** العنوان، السبب، المصدر، التاريخ، السمعة.

• **Block IP:** حظر عنوان جديد يدوياً.

• **Unblock:** فك الحظر وإزالة قاعدة nftables.

الحظر هنا فعلي على مستوى النواة: يُسَقَطُ ترافيك العنوان المحظور قبل وصوله لأي خدمة على الجهاز.

9.10 بيانات التدريب (training.html)

الغرض: أرشيف التنبيهات المحلولة لإعادة تدريب النموذج.

• **KPIs:** إجمالي السجلات، الهجمات المؤكدة، الإيجابيات الخاطئة.

• **الجدول:** النوع، التصنيف الحقيقي، الخطورة، الثقة، المصدر، البروتوكول، تكتيك MITRE، القرار، الإجراء، وقت الحل.

• **لوحة التفاصيل:** السياق الكامل للهجمة + القرار + الإجراء + الوصف.

• **Export CSV:** تصدير كل البيانات لإعادة التدريب.

9.11 التحليلات (analytics.html)

الغرض: مؤشرات تشغيلية واتجاهات.

• **KPIs:** متوسط زمن الحل (MTTR)، الإجمالي، اليوم الأكثر ازدحاماً، المهاجمون الفريدون.

• **الرسوم:** حجم التنبيهات اليومي، اتجاه الخطورة.

• **Top Attackers:** أكثر عناوين IP هجوماً.

MITRE ATT&CK (mitre.html) 9.12

الغرض: ربط الهجمات بإطار MITRE ATT&CK.

• **التكتيكات والتقنيات:** عرض التكتيك (Tactic) والتقنية (Technique) لكل نوع هجوم.

• **الخريطة:** ربط التنبيهات بمعرّفات MITRE مثل T1498.

9.13 التقارير (reports.html)

الغرض: إنشاء وتصدير تقارير الأمان.

• **Export CSV / Generate PDF:** تصدير التقارير.

• **KPIs:** إجمالي الحوادث، معدل الحظر، متوسط الاستجابة، التهديدات النشطة.

• **الرسوم:** اتجاهات الحوادث، توزيع التهديدات.

• **جدول التقارير الأخيرة:** الاسم، النوع، التاريخ، الحجم، الإجراءات.

9.14 الاستجابة (response.html)

الغرض: عرض خطة الاستجابة للحوادث.

• **خطوات تلقائية ويدوية:** ما نفّذه النظام وما يحتاج تدخّل المحلل.

• **الموافقات والخطوات التالية:** سير عمل الاستجابة.

9.15 التصنيف والتحليل (classification.html / analysis.html)

الغرض: عرض تصنيفات النموذج وتحليل مفضل للتدفقات.

• **classification:** نتائج التصنيف وتوزيع الأنواع.

• **analysis:** تحليل أعمق لميزات التدفق والتفسير.

9.16 المستخدمين (users.html)

الغرض: إدارة مستخدمي المنظمة.

• **Add User:** إضافة مستخدم جديد.

• **الجدول:** الاسم، البريد، الدور، المنظمة، الحالة، آخر دخول.

• **الإجراءات:** تعديل، حذف، إعادة تعيين كلمة المرور.

9.17 سجل التدقيق (audit.html)

الغرض: سجل ثابت لكل الإجراءات الحساسة.

• **عدادات:** دخول ناجح/فاشل، تجاوز حد الطلبات.

• **تصفية حسب الإجراءات:** LOGIN_OK, LOGIN_FAIL, ALERT_STATUS, FIREWALL_BLOCK ... إلخ.

• **الجدول:** الوقت، الإجراء، المنفذ، المورد، IP المصدر، التفاصيل.

Webhooks (webhooks.html) 9.18

الغرض: إرسال التنبيهات لأنظمة خارجية.

• **New Webhook**: إنشاء تكامل جديد.

• **الإعدادات**: الاسم، الرابط، القالب (Preset)، أدنى خطورة، التفعيل، السر (HMAC-SHA256).

• **الإجراءات**: تعديل، تدوير السر، حذف.

9.19 الإشعارات (notifications.html)

الغرض: عرض وإدارة الإشعارات.

• **Mark all read**: تعيين الكل كمقروء.

• **قائمة الإشعارات**: أيقونة حسب الخطورة، عنوان، وقت، زر انتقال.

9.20 الإعدادات (settings.html)

الغرض: تكوين المنصة (للمسؤول).

• **الأقسام**: إعدادات JWT، تحديد معدل الطلبات، GeoIP، البريد، إعادة محاولة Webhook، عتبة إعادة تدريب ML.

9.21 الاتصال (contact.html)

الغرض: التواصل مع فريق الدعم.

• **النموذج**: الموضوع، الرسالة، إرسال.

10. واجهات ال API الرئيسية

كل الطلبات تمرّ عبر المسار الأساسي `api/` وتتطلب توكن JWT (عدا الدخول والتسجيل).

الوظيفة	المسار	الطريقة
تسجيل الدخول وإصدار JWT	<code>api/auth/login/</code>	POST
قائمة التنبيهات (مع تصفية وترقيم)	<code>api/alerts/</code>	GET
تحديث حالة/بيانات تنبيه	<code>api/alerts/{id}/</code>	PATC H
رفع وتحليل ملف PCAP	<code>api/scan/</code>	POST
بدء المراقبة الحية	<code>api/monitor/start/</code>	POST
إيقاف المراقبة الحية	<code>api/monitor/stop/</code>	POST
حالة المراقبة	<code>api/monitor/status/</code>	GET
حظر عنوان IP	<code>api/firewall/block/</code>	POST
فك حظر عنوان IP	<code>api/firewall/{id}/</code>	DELET E
قائمة بيانات التدريب	<code>api/training-data/</code>	GET
تصدير بيانات التدريب CSV	<code>api/training-data//</code>	GET

الوظيفة	المسار	الطريقة
مؤشرات الأداء والاتجاهات	api/analytics/	GET
تصدير تقارير PDF	.../api/reports/	GET
طلب حساب مؤسسي	api/company-request/	POST

11. سيناريو عرض كامل (Demo)

سيناريو مقترح لعرض النظام من البداية للنهاية:

1.تفريغ البيانات للبدء من نظافة (alerts / training / blocked_ips).

2.تشغيل المراقبة الحية من صفحة **Monitoring** على بطاقة الشبكة.

3.تنفيذ هجمة من جهاز آخر (مثل ICMP Flood أو Port Scan).

4.ظهور التنبيه في صفحة **Alerts** مع تفاصيله.

5.حظر عنوان المهاجم عبر **Block IP**.

6.التأكد أن الترافيك توقّف فعلياً (ping يفشل من جهاز المهاجم).

7.تحوّل التنبيهات إلى **Resolved** تلقائياً.

8.ظهور السجل في **Training Data** مع القرار **BLOCKED**.

9.إصدار تقرير **PDF** للتوثيق.

Detected → Alert → Block IP → Traffic Dropped → Resolved → Archived → Report

12. التشغيل والإعداد

المنافذ الافتراضية:

العنوان	الخدمة
<code>http://127.0.0.1:5500</code>	Frontend
<code>http://127.0.0.1:8080</code>	Backend
<code>http://127.0.0.1:8001</code>	ML Service
<code>127.0.0.1:5432</code> قاعدة	PostgreSQL
<code>(ai_ids</code>	L

التحقق من الجاهزية:

```
curl http://127.0.0.1:8080/actuator/health # Backend
curl http://127.0.0.1:8001/health # ML Service
```

لتفعيل الحظر الفعلي على مستوى الشبكة، يجب منح صلاحية `nft` عبر `sudoers` كما في القسم 7.



الجزء الثاني

التوثيق التقني العميق

كيف يكتشف · كيف يربط · كيف يسجل · كيف يحظر

مبني على الكود الفعلي

التوثيق التقني العميق · كيف يكتشف النظام الهجمات، ويربطها، ويسجلها، ويحظرها · وثيقة دراسية مبنية على الكود الفعلي للنظام · الإصدار 3.0 | 2026

نظرة عامة على خط أنابيب المعالجة (Pipeline)

المرحلة 1 : التقاط الترافيك واستخراج التدفقات

المرحلة 2 : كيف يكتشف النظام الهجمات (طبقات الكشف)

2.1 الطبقة الأولى: قائمة التدفقات الآمنة

2.2 الطبقة الثانية: تصنيف الذكاء الاصطناعي (XGBoost)

2.3 الطبقة الثالثة: مرشحات الدقة

2.4 كشف ICMP الإرشادي (Heuristics)

2.5	كشف الشذوذ السلوكي (Behavioral Baseline)	•
-----	--	---

المرحلة 3	: تحديد الخطورة وربط MITRE ATT&CK
-----------	-----------------------------------

المرحلة 4	: نقل الكشف من ML إلى Backend
-----------	-------------------------------

المرحلة 5	: كيف تُربط الهجمات (Correlation / Incidents)
-----------	---

المرحلة 6	: كيف تُسجل التنبيهات (Persistence + Audit)
-----------	---

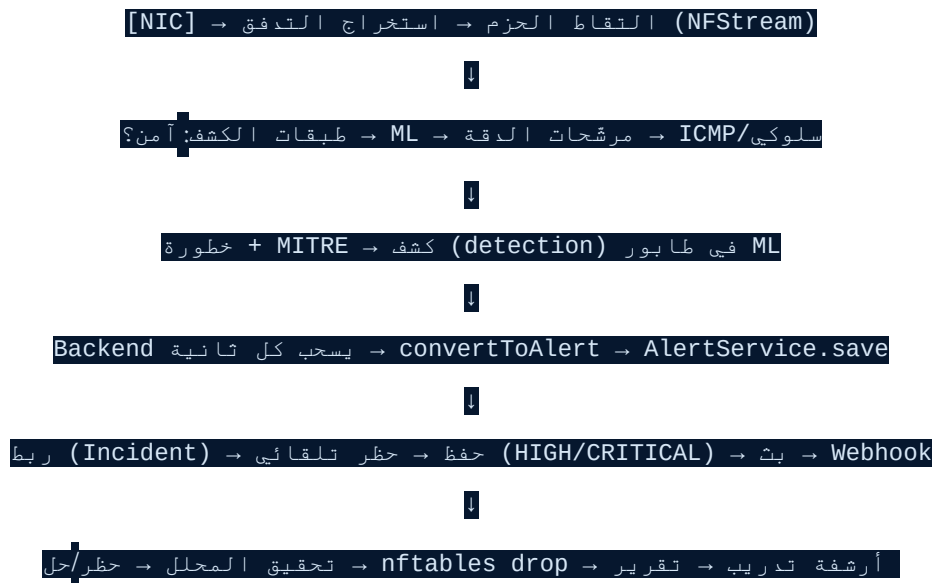
المرحلة 7	: كيف يعمل الحظر (Blocking + nftables)
-----------	--

المرحلة 8	: الحل الجماعي والأرشفة في التدريب
-----------	------------------------------------

ملخص الثوابت والمعاملات (Tunables)

1. نظرة عامة على خط أنابيب المعالجة

يمر كل تدفق شبكي عبر سلسلة مراحل واضحة من لحظة التقاطه حتى إصدار التقرير. هذه الوثيقة تتبع هذه الرحلة بالتفصيل التقني المبني على الكود الفعلي.



المكونات الأساسية في هذه الرحلة:

• **ML Service** (Python): `live_monitor.py` — الكشف والتصنيف.

• **Backend** (Java): `LiveMonitorService`, `AlertService`, `IncidentService`, `BlockedIpService`.

`.NftablesEnforcer`

2. المرحلة 1: التقاط الترافيك واستخراج التدفقات

تلتقط خدمة ML الحزم الخام من بطاقة الشبكة. ولأن الالتقاط يتطلب صلاحيات منخفضة المستوى، يُمنح مفسر Python القدرة `CAP_NET_RAW`

عبر السكريبت `scripts/grant-capture-perms.sh` دون تشغيل النظام كله بصلاحيات الجذر.

تُجمّع الحزم في **تدفّقات (Flows)** باستخدام مكتبة NFStream. التدفق هو محادثة شبكية بين طرفين، ويُستخرج منه عشرات الميزات

الإحصائية (عدد الحزم، البايتات، التوقيعات، أعلام TCP ...) إضافةً إلى بيانات وصفية:

```
_flow_meta(flow) → {  
  src_ip, dst_ip, src_port, dst_port, protocol,  
  app_name, app_category,                # NFStream nDPI  
  server_name (TLS SNI / HTTP Host),  
  ja3_client (JA3 fingerprint), ...  
}
```

المصدر: `ml-service/app/services/live_monitor.py → _flow_meta()`

3. المرحلة 2: كيف يكتشف النظام الهجمات

الكشف ليس قراراً واحداً، بل سلسلة طبقات تصفية تهدف إلى **دقة عالية وضوضاء منخفضة** (تقليل الإنذارات الكاذبة). تُنفَّذ كلها داخل

دالة `classify_batch()`.

2.1 الطبقة الأولى: قائمة التدفقات الآمنة

قبل استدعاء النموذج إطلاقاً، تُستبعد تدفقات معروفة أنها حميدة لتجنب إنذارات كاذبة، لأن مجموعة التدريب CICIDS2017 لم تتضمن بروتوكولات الشبكة المحلية:

• عناوين **Multicast** (224.0.0.0 – 239.255.255.255) و **IPv6 multicast** (ff00::/8).

• عناوين **Broadcast** (255.255.255.255).

• عناوين **Link-local** (169.254.0.0/16) ومنها نقطة بيانات السحابة 169.254.169.254.

• بروتوكولات اكتشاف محلية (mDNS, LLmNR, SSDP).

كل تدفق يطابق هذه القائمة يُصنّف **Benign** فوراً دون استنتاج ML.

المصدر: `_is_safe_flow() + SAFE_DST_EXACT`

2.2 الطبقة الثانية: تصنيف الذكاء الاصطناعي (XGBoost)

التدفقات غير الآمنة تُحوّل إلى مصفوفة ميزات رقمية بالترتيب نفسه الذي دُرّب عليه النموذج، ثم تُمرّر إلى نموذج **XGBoost**:

```
names = ml_service.feature_names
X = np.array([[float(f.get(n, 0.0)) for n in names] for f in feats])
ml_preds = ml_service._predict_array(X)
# كل تنبؤ: { predicted: "DDoS", confidence: 0.93, ... }
```

يُخرج النموذج لكل تدفق: التصنيف (نوع الهجوم أو Benign) ونسبة الثقة (0 إلى 1).

المصدر: `_classify_batch() + ml_service._predict_array()`

2.3 الطبقة الثالثة: مرشحات الدقة

حتى بعد أن يقول النموذج "هجوم"، تمر النتيجة عبر ثلاث بوابات صارمة قبل اعتمادها كتنبيه:

البوابة	الشرط	الفرض
مرشح ردّ	منفذ المصدر منفذ خدمة معروف والوجهة منفذ عابر ($\leq$)	تجاهل ترافيك ردود الخوادم الذي يخطئ النموذج في
الخادم	(1024	تصنيفه
بوابة الثقة	<code>confidence ≥ 0.80 (MIN_CONFIDENCE)</code>	إسقاط التنبؤات منخفضة الثقة
التحقق	≤ 2 تدفقات بنفس النوع من نفس المصدر خلال 60 ثانية	منع إنذار من تدفق شاذّ منفرد
بالتكرار		

إذا فشلت أي بوابة، يُحتسب التدفق Benign ولا يُنشأ تنبيه. هذه الطبقة هي سرّ انخفاض الإنذارات الكاذبة في النظام.

```
if confidence < MIN_CONFIDENCE:           # بوابة الثقة
    self._benign += 1; continue
hits = self._attack_hits[(src_ip, label)]  # بوابة التكرار
hits.append(now)
if len(hits) < MIN_REPEAT:
    continue
# فقط الآن يُعتبر هجوماً مؤكداً ←
```

المصدر: `_is_response_flow()` في `Layer 2/3` + `_classify_batch()`

2.4 كشف ICMP الإرشادي (Heuristics)

هجمات ICMP (مثل ping flood / sweep) لا يلتقطها النموذج جيداً، فيُستخدم منطق إرشادي يتجاوز النموذج ويُصدر تنبيهاً فورياً (دون انتظار

بوابة التكرار):

المعنى	القيمة	المعامل
عدد حزم في تدفق واحد يكفي لاعتباره فيضاناً	15	ICMP_FLOOD_PKTS
تدفقات ICMP لوجهة واحدة خلال النافذة → فيضان	3	ICMP_PROBE_THRESHOLD
عدد المضيفين الذين يلمسهم مصدر واحد → مسح	3	ICMP_SWEEP_HOSTS
النافذة الزمنية المنزلة للحساب	15	ICMP_WINDOW_SEC

المصدر: `_apply_icmp_heuristics()`

2.5 كشف الشذوذ السلوكي (Behavioral Baseline)

يقيس النظام خط أساس لمعدل التدفقات الطبيعي، فإذا قفز المعدل الحالي بمقدار معامل الطفرة فوق الأساس (وتجاوز 20 تدفق/دقيقة)، يُصدر

تنبيه Behavioral Anomaly بخطورة **MEDIUM** وتقنية T1046، حتى لو لم يتعرّف النموذج على نوع محدد:

```
if current_rate >= baseline_rate * BASELINE_SPIKE_FACTOR and current_rate > 20:
    confidence = min(0.95, 0.70 + 0.05 * (...))
    → { predicted: "Behavioral Anomaly", severity: "MEDIUM",
        mitre_technique: "T1046", tactic: "Discovery" }
```

(سطر 1262 ~) `live_monitor.py` المصدر: كشف الطفرة في

4. المرحلة 3: تحديد الخطورة وربط MITRE ATT&CK

بمجرد تحديد نوع الهجوم، تُربط به تلقائياً خطورته وتقنيته وتكتيك MITRE ATT&CK من جدول ثابت عبر الدالة `enrich(label)`:

الخطورة	التكتيك	التقنية	النوع
High	Credential Access	T1110	Brute Force
Medium	Discovery	T1046	Port Scan
	Impact	T1498	DDoS
High	Impact	T1498.001	ICMP Flood
Medium	Discovery	T1018	ICMP Sweep
High	Impact	T1499	DoS
High	Initial Access	T1190	Web Attack
High	Initial Access	T1190	SQL Injection
Critical	Execution	T1059	Command Injection
Critical	Impact	T1486	Ransomware

الخطورة	التكتيك	التقنية	النوع
Critical	Command and Control	T1105	Malware
Critical	Exfiltration	T1041	Data Exfiltration
High	Credential Access	T1557.002	ARP Spoofing

هذا الرابط هو ما يجعل كل تنبيه مفهوماً ضمن إطار معياري عالمي، ويُمكن صفحة [mitre.html](https://mitre.org/html) من عرض التكتيكات والتقنيات.

المصدر: ml-service/app/services/mitre.py → MITRE_MAPPING + enrich()

5. المرحلة 4: نقل الكشف من ML إلى Backend

الكشوفات المؤكدة تُوضَع في طابور `pending_` داخل خدمة ML. أما ال Backend فيسحبها دورياً:

- يفحص `LiveMonitorService.pollDetections()` كل ثانية واحدة حالة المراقبة.

- إن كانت تعمل، يسحب حتى 200 كشف عبر `monitorDrain(200)`.

- مع دفعة المصنّف كل 150ms في ML، يظهر الهجوم في الواجهة خلال ~1.2 ثانية من نهاية تدفّقه.

تُحوّل كل خريطة كشف إلى كيان `Alert` عبر `convertToAlert()`:

```
Alert a = new Alert();
a.setAttackType(d.get("predicted"));
a.setSourceIp(d.get("src_ip"));
a.setDestIp(d.get("dst_ip"));
a.setDestPort(...); a.setProtocol(...);
a.setConfidence(d.get("confidence"));
a.setSeverity(parseSeverity(d.get("severity")));
a.setMitreTechnique(d.get("mitre_technique"));
a.setMitreTactic(d.get("mitre_tactic"));
a.setStatus(AlertStatus.NEW);
alertService.save(a);
```

المصدر: `backend/.../service/LiveMonitorService.java`

مسار PCAP: تحليل الملفات يتبع نفس المنطق لكن عبر `ScanService` الذي يستدعي `bulkSaveFromScan()` لإدراج دفعة واحدة

بكفاءة.

6. المرحلة 5: كيف تُربط الهجمات (Correlation)

بدل إغراق المحلل بآلاف التنبيهات المنفردة، يجمع النظام التنبيهات المترابطة في **حوادث (Incidents)**. هذا يحاكي طريقة عمل مراكز العمليات الأمنية الناضجة.

قاعدة الربط (محافظة، دقة عالية):

عند كل تنبيه، تُستدعى `IncidentService.correlate()` التي تعمل بمنطق مفتاحين:

1. **المفتاح الأساسي:** نفس (عنوان المصدر + عائلة الهجوم + المنظمة) ضمن نافذة **ساعة واحدة** ← دمج (نفس المهاجم يكرر).

2. **المفتاح الموزع:** نفس (الوجهة + العائلة) بسابقة `dist` ← يُفَعَّل فقط لعائلات موزعة (DDoS/DoS/Brute/Flood/Scan) لرصد

هجوم من مصادر متعددة على هدف واحد.

عائلة الهجوم (Attack Family):

هي نوع الهجوم بعد إزالة اللاحقة الفرعية: `"DoS Hulk"` و `"DoS slowloris"` ← كلاهما عائلة `"DoS"`. هذا يجمع موجة هجمات متنوعة في سجل واحد.

مفتاح الربط (Correlation Key):

```
raw = srcIp + "|" + family + "|" + orgId
key = SHA-256(raw)[:40] // حرفاً سداً 40
```

عند الدمج (Rollup):

• زيادة عدّاد التنبيهات `++alertCount`.

• تحديث وقت آخر تنبيه `.lastAlertAt`.

• تصعيد الخطورة للأعلى إن كان التنبيه الجديد أشدّ.

يعمل correlate () في معاملة مستقلة REQUIRES_NEW حتى لا يُفسد تصادم مفتاح فريد (متوقع أثناء الإدراج المتزامن) معاملة حفظ

التنبيه بأكملها — وهذا يحمي مسح ملفات PCAP الكبيرة من الفشل الكامل.

المصدر: backend/.../service/IncidentService.java

7. المرحلة 6: كيف تُسجّل التنبيهات

كل التنبيهات تمرّ عبر نقطة دخول واحدة: `AlertService.save()`. خطواتها بالترتيب:

1. **فحص الحظر:** إن كان المصدر محظوراً مسبقاً، يُتجاهل التنبيه ويعود `null` (المحظور لا يظهر كهجمة جديدة).

2. **الربط:** استدعاء `incidents.correlate(a)` وربط الحادث قبل الحفظ.

3. **الحفظ:** `repo.save(a)` في جدول `alerts`.

4. **الحظر التلقائي:** إن كانت الخطورة **HIGH** أو **CRITICAL** يُستدعى `firewall.autoBlock()`.

5. **البث اللحظي:** `broadcaster.broadcast()` لتحديث الواجهة فوراً.

6. **Webhook:** إرسال للأنظمة الخارجية (fire-and-forget).

7. **إثراء GeoIP:** تعبئة بلد المصدر/الوجهة بشكل غير متزامن.

8. **سمعة IP:** تسخين ذاكرة Threat Intel.

```
public AlertDto save(Alert a) {
    if (firewall.isBlocked(a.getSourceIp(), orgId)) return null; // 1
    a.setIncident(incidents.correlate(a)); // 2
    Alert saved = repo.save(a); // 3
    if (severity == HIGH || severity == CRITICAL)
        firewall.autoBlock(saved, ...); // 4
    broadcaster.broadcast(dto, orgId); // 5
    webhooks.dispatchAlert(dto, orgId); // 6
    geoIp.enrichAsync(...); threatIntel.lookup(...); // 7,8
}
```

سجل التدقيق (Audit Log):

كل عملية حساسة تُسجّل في جدول `audit_log` عبر `audit_log(action, resource, id, details)`. أمثلة على الإجراءات:

`ALERT_STATUS`, `ALERT_UPDATE`, `FIREWALL_BLOCK`, `FIREWALL_AUTOBLOCK`, `FIREWALL_UNBLOCK`,

.ALERT_AUTO_RESOLVE

المصدر: backend/.../service/AlertService.java → save()

8. المرحلة 7: كيف يعمل الحظر

الحظر في MADRS ليس مجرد علامة في قاعدة البيانات — بل قاعدة فعلية في نواة لينكس تُسقط الحزم.

8.1 الحظر اليدوي (BlockedIpService.block)

1. **التطبيع:** `normalise()` تنظف العنوان وترفض المدخلات غير الصالحة.

2. **منع التكرار (Idempotency):** إعادة حظر عنوان محظور تُحدّث بياناته فقط دون صفّ مكرّر.

3. **حلّ التنبيهات:** `autoResolveAlerts()` تُحلّ كل تنبيهات هذا المصدر المفتوحة وتُلقح وسمّاً `[firewall:block#ID]`

بالوصف وتؤرشفها في التدريب.

4. **حلّ الحوادث:** `autoResolveIncidents()` تُغلق الحوادث المرتبطة.

5. **التطبيق الفعلي:** `enforcer.applyBlock(ip)` يدفع القاعدة لنواة النظام.

6. **التدقيق:** تسجيل `FIREWALL_BLOCK` مع عدد ما حلّ.

8.2 الحظر التلقائي (autoBlock)

يُستدعى تلقائياً للتنبيهات **HIGH/CRITICAL** فقط (الخطورات الأدنى مزعجة وقد تسبب حظراً خاطئاً). تتجاهل الدالة العناوين غير القابلة للحظر

عبر `isUnsafeForAutoBlock()`:

• Loopback (127.0.0.1), Link-local (169.254.0.0), Multicast/Broadcast.

• القيمة 0.0.0.0 أو MULTIPLE أو (fe80::) IPv6 link-local.

عناوين الشبكة الداخلية (RFC1918) **مسموح حظرها** لأن النظام يُستخدم غالباً لاحتواء مهاجم داخلي.

8.3 التطبيق على نواة النظام (NftablesEnforcer)

يُنشئ المُنفذ جدول `inet aisecc` ومجموعتين `aisecc_block_v4` و `aisecc_block_v6`, ثم قواعد `drop` على ثلاث سلاسل:

السلسلة	الاتجاه	الحالة المستهدفة
input	وارد من المهاجم	<code>ip saddr @set drop</code>
	صادر نحو الهدف	<code>ip daddr @set drop</code> (حين يكون جهازنا هو المهاجم)
forwar	ترافيك مُمرّر	عزل عبر ARP spoofing
d		

```
// تطبيق حظر عنوان :
nft add element inet aisecc aisecc_block_v4 { 192.168.1.110 }
// تمرر عبر nft كل أوامر : sudo -n /usr/sbin/nft ...
```

عند الإقلاع، يُزامن `syncEnforcerOnStartup()` النواة مع قاعدة البيانات: يُفرغ المجموعات ثم يعيد إضافة كل العناوين النشطة، فلا تبقى قواعد قديمة عالقة.

شُرط التنغيع: يحتاج مستخدم الـ Backend صلاحيات `sudoers`:

`/usr/sbin/nft` NOPASSWD: ALL=(root) mohamed

بدونها يسقط النظام إلى `NoOpFirewallEnforcer` (تسجيل فقط دون إسقاط فعلي).

المصدر: `backend/.../service/firewall/NftablesEnforcer.java + BlockedIpService.java`

9. المرحلة 8: الحل الجماعي والأرشفة في التدريب

الحل الجماعي (closeRelatedAlerts):

عند حل تنبيه أو وسمه إيجابية خاطئة، لا يُحلّ وحده. تبحث `closeRelatedAlerts()` عن كل التنبيهات المفتوحة بنفس (المصدر + النوع) في المنظمة، وتُغلقها جميعاً، وتؤرشفها، ثم تحذف صفوفها من قائمة التنبيهات النشطة:

```
related = repo.findOpenBySourceIpAndAttackType(srcIp, attackType, orgId, open);
for (Alert alert : related) {
    alert.setStatus(status);                // RESOLVED / FALSE_POSITIVE
    alert.setResolvedAt(resolvedAt);
    mlTraining.archiveAlert(saved);         // أرشفة في التدريب
    deleteAlertRow(saved);                 // إزالة من النشط
}
```

النتيجة: حل تنبيه واحد من 100 هجمة متكررة من نفس الجهاز والنوع يحلّها كلها ويؤرشفها دفعة واحدة.

الأرشفة في بيانات التدريب (MlTrainingService.archiveAlert):

تُخزّن في جدول `ml_training_data` مع السياق الكامل: النوع، الخطورة، الثقة، البلد، الوصف، تغذية ML، المسؤول، الحادث، إضافة إلى:

• **responseDecision:** يُشتقّ كالتالي:

• **BLOCKED** إذا احتوى الوصف وسم `[firewall:block#id]`.

• **MARKED_FALSE_POSITIVE** للإيجابيات الخاطئة.

• **RESOLVED** للتنبيهات المحلولة دون حظر.

• **responseAction:** وصف الإجراء المُتخذ.

هذه البيانات تُصدّر عبر CSV `MlTrainingController` وتُستخدم لإعادة تدريب النموذج — فيتعلم النظام من قرارات المحللين، ويفلق حلقة

الكشف ← الاستجابة ← التعلم.

المصدر: AlertService.closeRelatedAlerts() + MITrainingService.archiveAlert()

10. ملخص الثوابت والمعاملات (Tunables)

المعامل	القيمة	الموقع	الأثر
MIN_CONFIDENCE	0.80	live_monitor.py	أدنى ثقة لاعتماد تنبؤ ML
MIN_REPEAT	2	live_monitor.py	تكرارات لازمة لتأكيد الهجوم
REPEAT_WINDOW_SEC	60	live_monitor.py	نافذة احتساب التكرار
ICMP_FLOOD_PKTS	15	live_monitor.py	عتبة فيضان ICMP لتدقق واحد
ICMP_PROBE_THRESHOLD	3	live_monitor.py	تدفقات ICMP لوجهة → فيضان
ICMP_SWEEP_HOSTS	3	live_monitor.py	مضيفون لمصدر واحد → مسح
MERGE_WINDOW	1 ساعة	IncidentService.java	نافذة دمج التنبيهات في حادث
عتبة الحظر التلقائي	HIGH / CRITICAL	AlertService.java	الخطورات التي تُحظر تلقائياً
فترة سحب الكشوفات	1 ثانية	LiveMonitorService.java	تردد سحب Backend للكشوفات